



دانشگاه تهران

پردیس دانشکده های فنی

دانشکده مهندسی برق و کامپیوتر

گزارش پروژه پایانی

طراحی و ارزیابی سیاست مبتنی بر شبکه LSTM و آموزش با RL برای جایگزینی حافظه نهان در NDN

عنوان درس: مبانی رایانش توزیع شده

امین آقاکثیری – 810101381

بابک حسینی محتشم – 810101408

بهار 1405

فهرست مطالب

۱. مقدمه ۳

۲. مقدمه، مسئله و اهداف پروژه ۵

۳. مفاهیم NDN و حافظه نهان درون شبکه ۷

۴. معماری کلی سامانه و توپولوژی شبکه ۱۰

۵. جزئیات پیاده سازی شبیه ساز و حافظه نهان ۱۳

۶. تولید داده و آماده سازی مجموعه آزمایش ۱۶

۷. روش پیشنهادی و معماری مدل عصبی ۱۸

۸. آموزش نظارت شده، آموزش تقویتی و انتخاب مدل ۲۲

۹. نتایج آزمایش ۲۴

۱۰. تحلیل و تفسیر نتایج ۲۵

۱۱. جمع بندی ۲۶

۱. مقدمه

Named Data Networking (NDN) معماری شبکه را از ارتباط با میزبان به دریافت محتوا بر اساس نام تغییر می دهد. در این معماری، روترهای میانی می توانند بسته های Data عبوری را در حافظه ای به نام Content Store ذخیره کنند. اگر همان داده دوباره درخواست شود، یک روتر میانی می تواند پاسخ را پیش از رسیدن Interest به تولید کننده اصلی ارائه دهد. این قابلیت باعث کاهش بار تولید کننده، کاهش تعداد پرش های بازیابی و استفاده بهتر از پهنای باند می شود. با این حال، ظرفیت Content Store محدود است و هنگام ورود داده جدید باید مشخص شود کدام داده موجود حذف شود.

هدف این پروژه طراحی و پیاده سازی یک سیاست جایگزینی دقیق برای حافظه نهان روتر های NDN است. روش نهایی یک سیاست ترکیبی است و تنها یک شبکه LSTM ساده نیست. بخش اول آن تعداد درخواست های اخیر هر نام را در همان روتر اندازه گیری می کند و اثر درخواست های قدیمی را به مرور کاهش می دهد. بخش دوم (که بخش اصلی پروژه است)، مربوط به شبکه عصبی است که با استفاده از دو LSTM، ساختار سلسله مراتبی نام محتوا، تاریخچه زمانی درخواست های روتر و ویژگی های محلی هر عضو حافظه را تحلیل می کند. حاصل این دو بخش یک امتیاز نگهداری بین صفر و یک است. این امتیاز برای رتبه بندی نامزدهای حذف استفاده می شود و نباید بدون آزمون کالیبراسیون به عنوان احتمال دقیق درخواست دوباره تفسیر شود.

برای کاهش هزینه استنتاج در روتر، مدل همه ۱۶ عضو Content Store را در هر تصمیم بررسی نمی کند. ابتدا هشت عضو با کمترین تازگی دسترسی به روش LRU انتخاب می شوند و مدل فقط همین هشت نامزد را امتیازدهی می کند. در نتیجه تعداد اعضای امتیازدهی شده در هر تصمیم جایگزینی ۵۰ درصد کاهش می یابد. مدل روی hit معمولی یا هنگام وجود فضای آزاد در حافظه اجرا نمی شود.

برای چهار توپولوژی خطی، حلقه ای، ستاره ای و مش کامل، مدل های جداگانه آموزش داده شدند. دیتاست شامل ۲۰,۰۰۰ درخواست است که به ترتیب زمانی به ۱۵,۰۰۰ درخواست آموزش، ۲,۵۰۰ درخواست اعتبارسنجی و ۲,۵۰۰ درخواست آزمون دست نخورده تقسیم شد. همه بسته ها یک اندازه ثابت دارند و ظرفیت هر روتر ۱۶ بسته است. بنابراین نسبت برخورد درخواستی و نسبت برخورد بایتی در این آزمایش برابر هستند.

روش پیشنهادی با FIFO، LFU، LRU و Random در شرایط یکسان مقایسه شد. نسبت برخورد روش نهایی در توپولوژی های خطی، حلقه ای، ستاره ای و مش به ترتیب ۷۴.۶۴، ۷۴.۶۴، ۷۴.۹۶ و ۷۵.۰۰ درصد بود. میانگین نسبت برخورد روش پیشنهادی ۷۴.۸۱ درصد شد، در حالی که میانگین FIFO، LFU، LRU و Random به ترتیب ۷۱.۸۳، ۵۷.۷۱، ۶۸.۰۳ و ۶۹.۹۶ درصد بود.

۲. مسئله و اهداف پروژه

۲-۱. انگیزه پروژه

بخش بزرگی از ترافیک اینترنت شامل دریافت محتوا هایی است که چند کاربر بارها آنها را درخواست می کنند. در شبکه سنتی IP، درخواست به نشانی سرور ارسال می شود و محل قرارگیری داده اهمیت اصلی را دارد. در NDN، کاربر نام محتوا را اعلام می کند و شبکه می تواند پاسخ را از هر نسخه معتبر موجود در مسیر فراهم کند. این ویژگی، حافظه نهان درون شبکه را به یکی از بخش های اصلی معماری تبدیل می کند.

ذخیره سازی همه داده های عبوری امکان پذیر نیست. حافظه روتر محدود است و محتوای محبوب نیز در طول زمان تغییر می کند. اگر سیاست جایگزینی یک داده مفید را زود حذف کند، درخواست بعدی مجبور می شود مسیر طولانی تری تا یک کش بالادست یا تولید کننده طی کند. اگر داده ای را که دیگر تقاضا ندارد بیش از حد نگه دارد، فضای ارزشمند حافظه هدر می رود. بنابراین مسئله اصلی، تخمین ارزش آینده اعضای فعلی Content Store با استفاده از اطلاعات گذشته است.

۲-۲. صورت دقیق مسئله

فرض می کنیم هر روتر دارای Content Store با ظرفیت ثابت ۱۶ بسته است و اندازه تمام بسته ها برابر یک واحد است. هر Interest دارای نام دقیق محتوا و شرط MustBeFresh است. در صورت نبود پاسخ تازه در

کش، Interest طبق FIB به سمت تولید کننده ارسال می شود. Data روی مسیر معکوس باز می گردد و طبق سیاست Leave Copy Everywhere در روترهایی که miss داشته اند ذخیره می شود.

هنگامی که یک Data جدید به روتر می رسد، سه حالت وجود دارد. اگر داده از قبل در کش باشد، metadata آن به روز می شود. اگر فضای آزاد وجود داشته باشد، داده بدون حذف عضو دیگری درج می شود. اگر حافظه پر باشد، یک سیاست جایگزینی باید یکی از اعضای موجود را به عنوان قربانی انتخاب کند. روش این پروژه به هر نامزد یک امتیاز نگهداری می دهد و کم امتیازترین عضو را حذف می کند.

۳. مفاهیم NDN و حافظه نهان درون شبکه

۳-۱. تعریف نام ها

در NDN، واحد اصلی آدرس دهی نام است. یک نام از چند مولفه مرتب تشکیل می شود. برای نمونه، یک نام به صورت */dataset/item/category-03/000127* می تواند باشد. پیشوند های نام برای مسیریابی استفاده می شوند و ساختار سلسله مراتبی می تواند اطلاعات معنایی درباره نوع یا گروه محتوا در اختیار مدل قرار دهد.

پیاده سازی نام ها را به صورت URI متعارف دریافت، درصد ها را نرمال و مولفه های خالی را حذف می کند. برای جلوگیری از وابستگی واژگان به مجموعه آموزش، هر مولفه با BLAKE2 که یک الگوریتم hashing است، به یک شناسه پایدار در واژگان ۴۰۹۶ عضوی نگاشت می شود. شناسه PAD برای هم طول کردن دسته ها کنار گذاشته شده است. این روش اجازه می دهد نام های دیده نشده نیز در زمان آزمون رمز شوند.

۳-۲. بسته Interest و Data

مصرف کننده با یک بسته Interest نام داده مورد نظر را درخواست می کند. دو flag مهم در شبیه ساز CanBePrefix و MustBeFresh هستند. CanBePrefix مشخص می کند که پاسخ با نام طولانی تر ولی دارای پیشوند خواسته شده قابل قبول است یا نه. MustBeFresh تعیین می کند که یک نسخه منقضی شده نباید به درخواست پاسخ دهد. در نهایت، MustBeFresh برابر true و CanBePrefix برابر false است. بنابراین فقط پاسخ تازه با نام دقیق پذیرفته می شود.

بسته Data علاوه بر نام و اندازه، زمان تازگی یا FreshnessPeriod دارد. هنگام درج، زمان انقضا از زمان فعلی و FreshnessPeriod محاسبه می شود. یک عضو منقضی شده نمی تواند hit بشود و در زمان مناسب از حافظه کنار گذاشته می شود.

۳-۳. دو بخش اصلی روتر NDN

- **Content Store**: حافظه نهان بسته های Data. موضوع اصلی این پروژه سیاست حذف اعضای این بخش است.
- **Forwarding Information Base**: برای هر پیشوند نام، همسایه بعدی در مسیر تولید کننده را نگه می دارد.

۳-۴. روند کامل پاسخ یک درخواست

۱. Interest در روتر مصرف کننده وارد می شود و همان روتر ابتدا Content Store خود را بررسی می کند.

۲. اگر یک Data تازه و سازگار پیدا شود، درخواست با فاصله صفر از همان کش پاسخ داده می شود.

۳. در صورت miss، روتر بر اساس FIB همسایه بعدی را انتخاب می کند و Interest یک پرش جلو می رود.

۴. این روند تا اولین cache hit معتبر یا رسیدن به node-0 که تولید کننده است ادامه پیدا می کند.

۵. مسیر پیموده شده ثبت می شود و Data دقیقا روی مسیر معکوس بر می گردد.

۶. هر روتر مسیر که miss داشته است، Data را برای درج به سیاست کش خود تحویل می دهد.

۵-۳. سیاست جایگذاری

سیاست جایگذاری نسخه نهایی Leave Copy Everywhere است. هر روتر مسیر بازگشت که درخواست را از دست داده، یک نسخه از Data را ذخیره می کند. مزیت این روش سادگی و فراهم شدن نسخه های نزدیک به مصرف کننده است.

۴. معماری کلی سامانه و توپولوژی شبکه

۴-۱. اجزای سامانه

سامانه از چهار لایه اصلی تشکیل شده است. لایه داده، نام، Data، Interest و رویداد درخواست را نمایش می دهد. لایه شبکه، گراف روترها، FIB، ارسال Interest و مسیر بازگشت Data را مدیریت می کند. لایه کش شامل Content Store و سیاست های جایگزینی است. لایه یادگیری، ویژگی ها، دو LSTM، تابع امتیاز، pretrain، آموزش تقویتی و ذخیره مدل را در بر می گیرد.

۴-۲. گره ها و نقش آنها

در تمام آزمایش ها هشت روتر با نام node-0 تا node-7 وجود دارند. node-0 تولید کننده پیشوند */dataset/item* است و node-1 تا node-7 نقش مصرف کننده دارند. هر روتر یک Content Store مستقل با ظرفیت ۱۶ بسته، حالت پنهان LSTM و تاریخچه مستقل دارد. پارامترهای مدل درون یک توپولوژی بین روتر ها مشترک هستند، اما هیچ حالت اجرایی بین روتر ها مشترک نیست.

۴-۳. چهار توپولوژی

توپولوژی	نحوه اتصال	تعداد لینک
خطی	node-0 تا node-7 به صورت یک زنجیره متوالی	۷
حلقه ای	توپولوژی خطی به همراه لینک node-0 به node-7	۸
ستاره ای	node-1 مرکز غیر تولید کننده و هفت گره دیگر برگ هستند	۷
مش کامل	هر زوج از هشت روتر مستقیما متصل است	۲۸

۴-۴. ساخت FIB و انتخاب مسیر

پس از ساخت فهرست یال ها، پیاده سازی اتصال گراف را بررسی می کند و سپس برای هر منبع، کوتاه ترین مسیر بدون وزن تا تولید کننده را با جستجوی قطعی محاسبه می کند. ورودی FIB بر اساس پیشوند مولفه ای نام ساخته می شود و next hop همیشه یک همسایه فیزیکی است. اگر چند مسیر هم طول وجود داشته باشد، ترتیب قطعی گره ها نتیجه را ثابت نگه می دارد. این تصمیم برای تکرار پذیر بودن آزمایش ضروری است زیرا انتخاب تصادفی مسیر می تواند جریان محلی روتر ها و در نتیجه نتیجه کش را تغییر دهد.

۴-۵. دلیل آموزش جداگانه برای هر توپولوژی

یک درخواست خارجی لزوما در همه روتر های مسیر مشاهده نمی شود. اگر کش پاسخ بدهد، روترهای upstreaam آن Interest را نمی بینند. بنابراین دنباله درخواست محلی یک روتر به تصمیم های قبلی کش

و شکل گراف وابسته است. در توپولوژی خطی، یک Interest ممکن است از چند روتر عبور کند. در مش، معمولا فقط کش ورودی و تولید کننده در مسیر هستند. به همین دلیل، آموزش یک مدل مشترک و سپس ارزیابی آن روی همه گراف ها می تواند ساختارهای متفاوت مشاهده را نادیده بگیرد.

نسخه نهایی برای هر توپولوژی یک مدل، بهینه ساز، pretrain، RL، اعتبارسنجی و checkpoint مستقل دارد. تنها تنظیمات عمومی و seed اولیه مشترک هستند. داده آزمون هیچ توپولوژی در آموزش یا انتخاب مدل وارد نمی شود.

۵. جزئیات پیاده سازی شبیه ساز و حافظه نهان

۵-۱. ساختار فایل های کد

فایل	مسئولیت اصلی
names.py	نرمال سازی نام NDN، جدا کردن مولفه ها و نگاشت مولفه به شناسه.
packets.py	تعریف Interest، DataPacket و RequestEvent و کنترل معتبر بودن زمان، اندازه و تازگی.
store.py	Content Store مستقل از سیاست، lookup، درج، حذف، ظرفیت و FreshnessPeriod.
topology.py	ساخت گراف، کوتاه ترین مسیر، FIB، ارسال Interest، بازگشت Data و معیارهای شبکه.
features.py	آمار تقاضای محلی و ساخت هشت ویژگی عددی هر نامزد.
model.py	مدل های LSTM و Embedding.
agent.py	حالت هر روتر، فهرست کوتاه، انتخاب داده evict، درج و به روز رسانی RL.
training.py	تقسیم زمانی داده، برچسب استفاده بعدی، پیش آموزش و حلقه آموزش.
baselines.py	پیاده سازی Random، FIFO، LFU، LRU.
checkpoint.py	ذخیره وزن ها، تنظیمات کنترل سازگاری checkpoint.

۵-۲. Content Store

کلاس ContentStore فقط مسئول نگهداری داده و کنترل محدودیت ها است و درباره LRU یا مدل عصبی تصمیم نمی گیرد. هر CacheEntry بسته Data، زمان درج، آخرین زمان دسترسی و زمان انقضا را نگه می دارد. تابع lookup ابتدا سازگاری نام را بررسی می کند، سپس در صورت MustBeFresh معتبر بودن تازگی را می سنجد. تابع has_capacity ظرفیت تعدادی و در حالت عمومی ظرفیت بایتی را کنترل می کند. در تنظیم نهایی فقط محدودیت تعداد فعال است و هر بسته یک واحد حساب می شود.

جدا بودن Content Store از سیاست باعث می شود Random، FIFO، LFU، LRU و عامل عصبی دقیقا روی یک رفتار lookup و freshness اجرا شوند. همچنین تست های واحد می توانند منطق ظرفیت و انقضا را بدون وابستگی به مدل یادگیری بررسی کنند.

۵-۳. پردازش atomic یک Interest/Data

تابع پردازش شبیه ساز ابتدا مسیر تولید کننده را از FIB می گیرد و سپس روترها را از ورودی به سمت مبدا بررسی می کند. در هر روتر، begin_interest وضعیت قبل از پاسخ را ثبت و lookup را اجرا می کند. وقتی پاسخ یافت شد، مسیر رو به جلو متوقف می شود. سپس Data روی فهرست معکوس روترهای miss شده حرکت می کند و finish_interest آمار، حالت LSTM و در صورت نیاز تصمیم درج یا حذف را تکمیل می کند.

یک نکته مهم آموزش این است که کل رفت و برگشت یک درخواست، یک تراکنش atomic در نظر گرفته می شود. در غیر این صورت ممکن بود وزن مدل در میانه پردازش یک درخواست تغییر کند و روترهای مختلف همان Data با دو نسخه پارامتر تصمیم بگیرند.

۵-۴. حالت محلی روتر

برای هر node یک NodeState جدا ساخته می شود. این حالت شامل Content Store، RequestStatistics، حالت hidden و cell در LSTM دوم، معیارهای محلی و RL است. دسترسی به این حالت با شناسه node انجام می شود و reset_episode همه حالت ها را پیش از اجرای مستقل یک سیاست پاک می کند. تست های جداسازی چند گره بررسی می کنند که درخواست یک روتر، hidden state یا شمارنده روتر دیگر را تغییر ندهد.

۵-۵. کنترل تازگی و داده ورودی

در زمان بازگشت Data، agent ابتدا سازگاری نام پاسخ با Interest را کنترل می کند. گزینه bypass در کد وجود دارد، ولی در تنظیم نهایی غیرفعال است. بنابراین Data ورودی همیشه پذیرفته می شود و در صورت پر بودن حافظه، یکی از اعضای موجود حذف می گردد.

۶. تولید داده و آماده سازی مجموعه آزمایش

۶-۱. فضای نام و کاتالوگ

کاتالوگ شامل ۳۰۰ شی یکتا است. اشیا در هشت دسته سلسله مراتبی قرار می گیرند و نامی مانند */dataset/item/category-03/000127* دارند. هر شی دقیقا یک بسته و اندازه آن یک واحد است. هر شی در زمان ساخت کاتالوگ یک FreshnessPeriod ثابت دریافت می کند که از توزیع یکنواخت بین ۲ و ۳۰۰ نمونه گیری می شود. درخواست های بعدی همان شی از همان مقدار استفاده می کنند. زمان ورود درخواست ها با فاصله نمایی و نرخ ۲۰ درخواست در هر واحد زمان جلو می رود.

۶-۲. محبوبیت Zipf

محبوبیت پایه محتوا از توزیع Zipf با آلفای ۱.۱ پیروی می کند. در این توزیع، تعداد کمی از اشیا درخواست زیادی دارند و یک دنباله بلند از اشیای کم محبوب وجود دارد. این رفتار به ترافیک محتوایی نزدیک تر از توزیع یکنواخت است. احتمال انتخاب رتبه r به صورت متناسب با $(\frac{1}{r})^{1.1}$ محاسبه و سپس نرمال می شود.

۶-۳. محلی بودن زمانی

فقط محبوبیت سراسری برای ساخت رفتار واقعی کش کافی نیست. کاربران معمولا در بازه کوتاه چند بار به یک محتوا یا مجموعه مرتبط بر می گردند. برای هر مصرف کننده یک پنجره ۶۴ عضوی از درخواست های اخیر

نگهداری می شود. با احتمال ۰.۴۵، نام بعدی از این پنجره انتخاب می شود و وزن انتخاب با سن درخواست با ضریب ۰.۸۸ کاهش می یابد. بنابراین رخدادهای تازه تر احتمال بیشتری دارند. با احتمال باقی مانده، نام از توزیع Zipf فعلی نمونه گیری می شود.

۵-۶. تقسیم زمانی داده

بخش	تعداد درخواست	درصد	کاربرد
آموزش	۱۵۰۰۰	۷۵٪	ساخت برچسب آینده فقط در همین بخش، پیش آموزش و RL.
اعتبارسنجی	۲۵۰۰	۱۲.۵٪	مقایسه checkpoint اولیه، نظارت شده و RL و انتخاب بهترین مرحله.
آزمون	۲۵۰۰	۱۲.۵٪	ارزیابی نهایی یک بار پس از قفل شدن انتخاب مدل.

۷. روش پیشنهادی و معماری مدل عصبی

۷-۱. ایده اصلی

روش نهایی از دو بخش ساخته شده است. بخش اول یک رتبه بندی ساده، علی و قابل اعتماد بر اساس محبوبیت اخیر است. بخش دوم یک شبکه عصبی است که ساختار نام، تاریخچه درخواست روتر و ویژگی های محلی را بررسی می کند و فقط یک اصلاح کوچک به رتبه پایه اضافه می کند. این طراحی از یک طرف اجازه یادگیری الگوهای پیچیده را می دهد و از طرف دیگر مانع می شود یک شبکه آموزش دیده ناپایدار، ترتیب قوی محبوبیت اخیر را به طور کامل از بین ببرد.

۷-۲. ویژگی های عددی هر نامزد

شماره	ویژگی	معنا
۱	decayed frequency	تعداد موثر درخواست های اخیر نام با فراموشی نمایی.
۲	request recency	فاصله از آخرین درخواست محلی نام.
۳	regularity	پایداری یا پراکندگی فاصله های بین درخواست ها.
۴	cache-hit fraction	نسبت دفعاتی که درخواست نام در همان روتر hit بوده است.
۵	residence recency	زمان سپری شده از درج یا آخرین دسترسی عضو Content Store.
۶	freshness remaining	مقدار FreshnessPeriod که هنوز منقضی نشده است.
۷	relative size	اندازه بسته نسبت به ظرفیت.
۸	incoming/resident	تفاوت Data ورودی و عضو resident.

۷-۳. رمزگذار نام

نام به حداکثر ۱۶ مولفه تبدیل می شود. هر شناسه مولفه یک embedding با بعد ۳۲ دارد. LSTM اول دنباله embedding ها را می خواند و آخرین حالت پنهان ۴۸ بعدی آن نمایش نام است. برای دسته ای از نام های با طول متفاوت، *pack_padded_sequence* استفاده می شود تا padding وارد حالت نهایی نشود.

۷-۴. LSTM تاریخچه روتر

هر بار یک Interest در روتر مشاهده می شود، نمایش ۴۸ بعدی نام درخواست شده به LSTM دوم داده می شود. این LSTM یک لایه و hidden size برابر ۶۴ دارد. خروجی آن خلاصه ای از دنباله زمانی درخواست های همان روتر است. وزن های LSTM بین روترهای یک توپولوژی مشترک هستند، ولی زوج hidden و cell برای هر روتر جدا نگهداری می شود.

۷-۵. head امتیازدهی

برای هر نامزد، بردار نام ۴۸ بعدی، زمینه روتر ۶۴ بعدی و هشت ویژگی عددی به یک بردار ۱۲۰ بعدی تبدیل می شوند. این بردار از یک لایه Linear با خروجی ۶۴، Dropout، GELU، LayerNorm صفر و یک Linear نهایی با خروجی یک عبور می کند. کل مدل ۱۸۳۹۴۵ پارامتر دارد که ۱۸۳۹۳۷ عدد قابل آموزش هستند.

۷-۶. امتیاز نهایی

logit نهایی نامزد i به شکل زیر محاسبه می شود:

$$z_i = p_i + 0.05 \tanh(\Delta z_\theta(i, h_v, x_i))$$

$$s_i = \text{sigmoid}(z_i)$$

تابع tanh اصلاح یادگرفته شده را محدود می کند و ضریب ۰.۰۵ باعث می شود دامنه اثر آن حداکثر مثبت یا منفی ۰.۰۵ باشد. امتیاز s بین صفر و یک قرار دارد. امتیاز بالاتر به معنای تمایل بیشتر به نگهداری است.

استفاده از sigmoid فقط خروجی را محدود می کند و به تنهایی کالیبراسیون احتمال را تضمین نمی کند.

بنابراین در گزارش از عبارت امتیاز نگهداری استفاده می شود، نه احتمال دقیق درخواست مجدد.

۸. آموزش نظارت شده، آموزش تقویتی و انتخاب مدل

۸-۱. دلیل آموزش دو مرحله ای

شروع مستقیم RL با پاداش صفر و یک می تواند واریانس زیادی داشته باشد زیرا اثر یک تصمیم حذف ممکن است چند درخواست بعد دیده شود. pretrain ابتدا مدل را به سمت یک ترتیب مفید هدایت می کند. سپس RL با اجرای واقعی کش، اثر تصمیم ها بر hit های آینده را اصلاح می کند.

۸-۲. ساخت جریان مشاهده محلی

برای هر توپولوژی، LRU فقط بخش ۱۵۰۰۰ درخواستی آموزش را اجرا می کند. این replay مشخص می کند هر Interest واقعا به کدام روترها می رسد و دنباله محلی هر روتر را می سازد. تعداد مشاهدات روترها از تعداد درخواست خارجی بیشتر است، زیرا یک Interest می تواند در چند روتر miss شود.

توپولوژی	مشاهده روتر ساخته شده از ۱۵۰۰۰ درخواست
خطی	۲۸۹۲۹
حلقه ای	۲۴۳۶۲
ستاره ای	۲۲۶۷۸
مش	۱۹۷۵۶

۸-۳. برچسب استفاده بعدی

برای هر نامزد i فاصله d تا درخواست دقیق بعدی همان نام در جریان محلی همان روتر بررسی می شود. افق H برابر ۳۲ مشاهده است. اگر استفاده بعدی در این افق باشد، مطلوبیت برابر معکوس فاصله است. در غیر این صورت مطلوبیت صفر در نظر گرفته می شود:

$$u_i = 1/d_i \text{ if } d_i \leq 32, \text{ otherwise } u_i = 0$$

این هدف احتمال را پیش بینی نمی کند. فقط می گوید کدام نامزد باید بالاتر رتبه بندی شود. برای هر زوج با $u_i > u_j$ زیان زیر کمینه می شود:

$$L_{rank} = \text{softplus}(-(z_i - z_j))$$

زیان پیش از حذف روی فهرست کامل نامزدها، شامل عضوی که بعدا evict می شود، محاسبه می گردد.

۸-۴. تعریف پاداش RL

اگر یک درخواست در Content Store محلی روتر hit شود پاداش ۱ و در صورت miss پاداش صفر است.

$$r_t = 1 \text{ for local cache hit, } 0 \text{ otherwise}$$

اما زمان بندی پاداش اهمیت دارد. تصمیم حذف پس از پردازش درخواست t گرفته می شود. پس نمی تواند hit همان درخواست را ایجاد کرده باشد. عمل پس از t باید از پاداش های آینده اعتبار بگیرد:

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots$$

۹. نتایج آزمایش

توپولوژی	سیاست	Hit Ratio	مراجعه به producer	میانگین hop
خطی	**روش نهایی**	**۷۴.۶۴٪**	**۶۳۴**	**۰.۸۶۹۲**
خطی	LRU	۷۰.۳۲٪	۷۴۲	۰.۹۵۳۲
خطی	LFU	۶۲.۴۰٪	۹۴۰	۱.۳۴۴۸
خطی	FIFO	۶۵.۳۲٪	۸۶۷	۱.۰۶۷۶
خطی	Random	۷۰.۶۴٪	۷۳۴	۱.۰۴۳۲
حلقه ای	**روش نهایی**	**۷۴.۶۴٪**	**۶۳۴**	**۰.۶۲۹۲**
حلقه ای	LRU	۷۲.۰۴٪	۶۹۹	۰.۶۵۴۰
حلقه ای	LFU	۵۸.۰۸٪	۱,۰۴۸	۱.۰۱۲۸
حلقه ای	FIFO	۶۸.۸۰٪	۷۸۰	۰.۷۳۵۶
حلقه ای	Random	۷۰.۲۴٪	۷۴۴	۰.۷۶۸۸
ستاره ای	**روش نهایی**	**۷۴.۹۶٪**	**۶۲۶**	**۰.۵۰۶۴**
ستاره ای	LRU	۷۱.۲۴٪	۷۱۹	۰.۵۲۷۲
ستاره ای	LFU	۵۵.۴۸٪	۱,۱۱۳	۰.۸۲۸۸
ستاره ای	FIFO	۶۷.۰۸٪	۸۲۳	۰.۵۸۸۴
ستاره ای	Random	۶۹.۶۸٪	۷۵۸	۰.۶۲۷۶
مش	**روش نهایی**	**۷۵.۰۰٪**	**۶۲۵**	**۰.۳۱۷۲**
مش	LRU	۷۳.۷۲٪	۶۵۷	۰.۳۱۸۸
مش	LFU	۵۴.۸۸٪	۱,۱۲۸	۰.۵۲۰۰
مش	FIFO	۷۰.۹۲٪	۷۲۷	۰.۳۵۸۴
مش	Random	۶۹.۲۸٪	۷۶۸	۰.۳۸۶۰

۱۰. تحلیل و تفسیر نتایج

روش پیشنهادی در توپولوژی های خطی، حلقه ای، ستاره ای و مش به ترتیب نسبت برخورد ۷۴،۶۴، ۷۴،۶۴، ۷۴،۹۶ و ۷۵ درصد را به دست آورد و در هر چهار توپولوژی بهتر از روش های پایه عمل کرد. میانگین نسبت برخورد آن ۷۴،۸۱ درصد بود که ۲،۹۸ واحد درصد بیشتر از LRU به عنوان بهترین روش پایه است.

بیشترین برتری در توپولوژی خطی مشاهده شد، زیرا مسیرهای طولانی تر باعث می شوند تصمیم مناسب کش تاثیر بیشتری بر کاهش انتقال درخواست داشته باشد. در مقابل، کمترین اختلاف در توپولوژی مش بود زیرا مسیرها کوتاه تر هستند و LRU نیز در چنین شبکه ای عملکرد خوبی دارد. یکسان بودن نسبت برخورد در خط و حلقه همراه با کمتر بودن میانگین پرش در حلقه نیز نشان می دهد که ساختار شبکه علاوه بر سیاست کش بر هزینه دریافت داده تاثیر دارد.

LFU به دلیل وابستگی زیاد به محبوبیت گذشته، در برابر تغییر محبوبیت داده ها ضعیف عمل کرد. FIFO هم تازگی و تعداد درخواست ها را در نظر نمی گیرد. Random به دلیل وجود محبوبیت Zipf و محلی بودن زمانی درخواست ها نتیجه نسبتا خوبی گرفت اما همچنان از روش پیشنهادی ضعیف تر بود.

روش پیشنهادی علاوه بر افزایش نسبت برخورد، میانگین مراجعه به تولید کننده را از حدود ۷۰۴ در LRU به حدود ۶۳۰ کاهش داد که تقریبا ۱۰،۶ درصد کاهش است. این روش همچنین کمترین میانگین پرش را در همه توپولوژی ها ثبت کرد. البته روش نهایی ترکیبی از محبوبیت اخیر، LSTM، pretrain و RL است و تمام بهبود نتایج را نمی توان فقط به RL نسبت داد.

۱۱. جمع بندی

در این پروژه یک شبیه ساز چند روتره NDN و یک سیاست جایگزینی ترکیبی طراحی و پیاده سازی شد. شبیه ساز نام، FIB، Content Store، FreshnessPeriod، Data، Interest، کوتاه ترین مسیر، بازگشت Data و Leave Copy Everywhere را پشتیبانی می کند. چهار توپولوژی خطی، حلقه ای، ستاره ای و مش با هشت روتر ارزیابی شدند و هر روتر حافظه مستقل ۱۶ بسته ای داشت.

روش پیشنهادی تقاضای اخیر هر نام را با کاهش نمایی و ثابت زمانی ۳۲ دنبال می کند. مدل عصبی از یک LSTM نام (همان LSTM اول) با hidden size برابر با ۴۸، یک LSTM تاریخچه (همان LSTM دوم)، روتر با hidden size برابر با ۶۴ و هشت ویژگی عددی استفاده می کند.

برای عملی شدن تصمیم در روتر، فقط هشت عضو قدیمی تر از میان ۱۶ resident امتیازدهی می شوند. مدل روی hit یا درج در حافظه غیر پر اجرا نمی شود. آموزش برای هر توپولوژی جدا انجام شد و شامل pretrain، REINFORCE با پاداش hit آینده و انتخاب checkpoint فقط با اعتبارسنجی بود.

در آزمون، روش نهایی در توپولوژی خطی، حلقه، ستاره و مش به ترتیب ۷۴.۶۴، ۷۴.۶۴، ۷۴.۹۶ و ۷۵.۰۰ درصد hit به دست آورد و در هر چهار توپولوژی از FIFO، LFU، LRU و Random بالاتر بود. میانگین آن ۷۴.۸۱ درصد و ۲.۹۸ واحد درصد بیشتر از میانگین LRU بود. روش نهایی همچنین کمترین مراجعه به تولید کننده و کمترین میانگین پرش را در همه گراف ها ثبت کرد.